Checklist del backend FST en Spring Boot

Sale de `INSTRUCCIONES-WEB.md` (guía del proyecto TT 2026-B066), traducido de Flask a Spring Boot. Sigue el orden de construcción de la sección 12 de la guía. Entre paréntesis va la sección o regla de origen.

No cubre el frontend ni el SQL y Docker de los anexos. Las decisiones de integración con el frontend del 15-oct-2026 están reflejadas abajo y explicadas en `PARA_VANESA_API.md` (rama `feat/frontend-v2-servicios`). Lo que Vanesa propuso después (18-oct-2026) está en la sección 12.

Cómo leerlo: `[x]` = hecho y verificado en el código. `[ ]` = pendiente. Si una casilla vacía trae una nota "hecho:", ya existe una parte y la nota dice qué falta.

Última revisión del estado (25-oct-2026, versión 0.86.0): el backend compila y corre en Docker junto con la base, el worker real (modelo `fst_cnn_seg.pt`) y el frontend de Vanesa; el compose definitivo está en la raíz (proyecto `fst-portable`, se levanta con `portable\iniciar.bat`). Probado con un video real (`AcuBN1.MOV`, 18-oct) y por Vanesa en el navegador contra nuestro backend (20-oct, `e66ce40`). **Hecho el 25-oct:** la pausa de R-15 (§6 y §10, encargo 32), probada en desarrollo con una pausa forzada; y el encargo 33 (foto de la pausa igual que fst_auto, «recuadro casi igual», botón «El recuadro está bien, continuar» y editar experimento solo el creador o un Administrador), en código con pruebas unitarias y sin probar contra el stack; y el encargo 34 (reportes guardados en `REPORTE` y su invalidación, §8), igual; y el encargo 35 (administración y recuperar contraseña, §2, §3 y §11), con pruebas JUnit contra una PostgreSQL de prueba y sin probar con el correo real ni el navegador; y el encargo 36 (reiniciar un análisis, §6), con pruebas JUnit contra una PostgreSQL de prueba y sin probar con el worker real; y el encargo 37 (bloques de 5 s y pruebas JUnit de las reglas del video, su token, la revisión y `PUT .../segundos`, §7), sin probar contra el front. Erick prueba los cinco en su máquina. **Sin hacer:** CSV y XLSX (§8), el correo de cada notificación (§9) y el `error_log` de nginx con `?vt=` (§7).

0. Base y Docker

☑ `docker compose up` levanta `db`, `backend`, `worker` y `mailhog` (dev), con volúmenes `videos_reportes` y `pgdata` (§2)
☑ `.env.example` con todas las variables de §2, más `VIDEO_TOKEN_MINUTES` (120). Las de autenticación y `ADMIN_*` el backend las lee desde la sección 2
☑ `application.properties` lee de variables de entorno lo que existe hoy (DB, SMTP, límite de subida)
☑ Endpoint `/health`
☑ Swagger con todas las rutas: sin `paths-to-match`, springdoc documenta también las que se agreguen (verificado en `/v3/api-docs`)
☑ CORS para el frontend (`localhost:5173`)
☑ Rutas guardadas en la BD relativas a la raíz del volumen, que en el contenedor es `/data/archivos` (§2): `videos/{idVideo}.{ext}`
☑ Límite de subida configurable, y `413` si se excede (§11.5): `ManejadorErrores` responde `{data: null, error, status: 413}`; probado con `MAX_UPLOAD_MB=1` y un archivo de 3 MB
☑ ffmpeg en el Dockerfile del backend (hace falta para §5); `ffprobe -version` corre en el contenedor
☑ Puerto del backend `8084`, dentro y fuera de Docker: el `8080` lo ocupa Traefik de CVAT en esta máquina. Vanesa ya lo usa (`VITE_API_BASE=http://localhost:8084` en `PARA_ERICK.md`)
☐ `docker-compose.yml` definitivo en la raíz (commit `99a016d`, 23-oct): proyecto `fst-portable`, servicios `db`, `api`, `worker` y `web`, que se levanta con `portable\iniciar.bat`; el de desarrollo sigue en `backend-spring/`. Hecho: verificado solo con `docker compose config` (rutas, volúmenes y variables); falta probarlo en la otra PC

1. Base de datos

☑ SQL del Anexo B con los dominios cerrados, `UNIQUE` y llaves foráneas `NO ACTION` (§5), incluido `tipo_conducta`
☑ `ddl-auto=validate`
☑ 12 entidades, sin `Conducta`, `Intervalo` ni `Presenta` (§4.9, D-102)
☑ `ANALISIS.idVideo` único
☑ `rutaDiagnostico` solo se llena si `estado = 'error'` (`CHECK` en el SQL)
☑ Cardinalidades validadas en el servicio, sin `CHECK` de conteo: tanda de 2 a 4 especímenes, grupo con 1 o más tandas, mínimo 2 especímenes por grupo (§5). Tanda de 2 a 4: `VideoServiceImpl` rechaza `nCilindros` fuera de ese rango y crea exactamente esos especímenes. Grupo con 1 o más tandas: un grupo solo nace dentro de la misma transacción que su primera tanda (`resolverDestino`; si algo falla se deshace todo, revisado en el código) y, desde el 23-oct, al eliminar su última tanda el grupo se borra en la misma transacción (probado en desarrollo: la única tanda de un grupo → `204` y el grupo desaparece; una de dos → el grupo se conserva). Mínimo 2 especímenes por grupo: sale de las dos anteriores
☑ Contraseñas con bcrypt, nunca en texto plano: `BCryptPasswordEncoder` en el login y en `AdminInicial`, y el hash con `@JsonIgnore`
☑ Valores de dominio como enums de Java en lugar de `String` (23-oct): los 9 dominios de la base tienen su enum en `domain.valores` (`TipoGrupo`, `SesionVideo`, `FormatoReporte`, `TipoNotificacion`, `EstadoAnalisis`, `EtapaAnalisis`, `NivelClasifAnalisis`, `OrigenSegundo`, `TipoConducta`), con el texto exacto de la base (`valor()`, `desdeValor`, `@JsonValue`) y un conversor JPA por enum en las entidades, que conservan su `columnDefinition`. Las listas que se validaban a mano salen de los enums. Probado en desarrollo: el backend arranca con `ddl-auto=validate`; `GET /experiments/{id}`, `/status`, `/results` y `/results/byminute` devuelven los mismos textos que la base; y borrar un experimento con reporte y notificaciones (lectura de entidades por los conversores) da `204`. No se probó la lectura de `Segundo.conducta` nula ni de `Analisis` por los conversores: ningún servicio lee esas entidades hoy (todo va por SQL directo)

2. Autenticación

☑ Spring Security con JWT de larga duración, configurable con `JWT_EXPIRES_HOURS` (§11.1): Nimbus HS256 con `tipo = sesion`, token solo en la cabecera (en la URL o alterado da `401`), y `ConvertidorJwt` revisa en la base en cada petición si la cuenta sigue activa (desactivada da `401` aunque el token no haya vencido) y si es Administrador. No arranca con un `JWT_SECRET` de menos de 32 caracteres
☑ `POST /auth/login` con error `401` genérico (R-01), el mismo con contraseña mala, correo inexistente o cuenta inactiva. La respuesta trae `token`, `idInstitucional`, `nombre`, `apellidos`, `correo`, `rol` y `cambioRequerido`, sin el hash
☑ `POST /auth/logout`, sin lista de bloqueo: pide token (§6) y responde `200`
☑ `PATCH /auth/password`, y el cambio es obligatorio si `cambioRequerido`: mientras no la cambie, todo salvo esta ruta y el logout responde `403`; mínimo 8 caracteres y máximo 72 bytes (límite de bcrypt)
☑ `POST /auth/recover` envía un enlace JWT firmado por correo, con vigencia `RESET_LINK_MINUTES` (60 por omisión). Responde `200` igual exista o no el correo (o la cuenta esté desactivada) y el correo sale en otro hilo. Probado con JUnit contra una PostgreSQL de prueba (encargo 35); falta probarlo con el correo real y el navegador
☑ `POST /auth/reset {token, contrasenaNueva}`: el enlace sirve una sola vez y sin tablas nuevas (el token lleva una huella HMAC del hash de la contraseña; al cambiar esta deja de servir, también por el cambio normal o por una temporal nueva). `400` si venció, ya se usó o no es un enlace; `422` si la contraseña no cumple (no gasta el enlace); dos usos a la vez, solo pasa uno. Probado con JUnit (encargo 35)
☑ `PATCH /users/me` (perfil): nombre, apellidos y correo, `409` si el correo es de otra cuenta

☑ Todo exige JWT salvo login, recover, reset, `/health` y Swagger: sin token se responde `401` con envoltorio; CORS centralizado y el preflight de `localhost:5173` pasa
☑ Respuesta con envoltorio `{data, error, status}`, salvo las rutas que devuelven archivo (§6): `Respuesta.ok/error` en `/experiments` y `/health`, y `401`/`403` de Spring Security escritos con la misma forma
☑ Manejo de errores global con `@RestControllerAdvice` (`ManejadorErrores` + `ErrorApi`) en lugar de mapas armados a mano: 400, 404, 405, 409, 413, 415 y 500, con el mensaje en `error`, sin `trace` y sin la causa en los 500
☑ Comando para crear el primer administrador con `ADMIN_*`, que ponga `cambioRequerido = true` y lo inserte en `ADMINISTRADOR` (§11.7): `AdminInicial` al arrancar; idempotente (al reiniciar dice "ya existe" y no duplica), sin escribir la contraseña en el log; con `ADMIN_ID` vacío solo avisa

3. Administración (§6, R-07, R-16)

☑ Verificación de rol Administrador en el backend en cada `/admin/*`, con `403` si no lo es (`SeguridadConfig`, el rol se lee de la base en cada petición). Probado con JUnit (encargo 35)
☑ `GET /admin/users?rol=&estado=` (`Investigador`/`Administrador`, `activa`/`inactiva`; `400` con otro valor); trae `nExperimentos`. Probado con JUnit
☑ `POST /admin/users` genera una contraseña temporal (12 caracteres, `SecureRandom`), la envía por correo después del commit, la devuelve una sola vez en `passwordTemporal` y pone `cambioRequerido = true`; con ella solo se puede cambiar la contraseña. `POST /admin/users/{id}/reset-temporal` hace lo mismo para una cuenta existente (la anterior y los enlaces pendientes dejan de servir). Probado con JUnit; el correo real no
☑ `409` si el identificador o el correo ya existen (también con dos peticiones a la vez). Probado con JUnit
☑ `PATCH /admin/users/{id}` modifica nombre, apellidos, correo y rol, o desactiva con `activo = false`; no hay `DELETE`. El identificador no se cambia (`400` si llega uno distinto): es la llave de la cuenta. Si cambia el nombre o los apellidos invalida los reportes de la cuenta, igual que `PATCH /users/me`. Probado con JUnit
☑ `409` al desactivar la última cuenta de Administrador activa o quitarle el rol (R-16): `ReglaAdministradores`, con la parte pura probada sola y las filas de `ADMINISTRADOR` bloqueadas en orden, así que dos peticiones a la vez no dejan el sistema sin ninguno (25 vueltas de cada caso; sin el candado fallan). Candados de cuenta con `FOR NO KEY UPDATE` (`CandadosDeCuenta`), también en `PATCH /users/me`
☑ `GET /admin/system` con disco y cola: `disco {usado_gb, total_gb, videos_gb, resultados_gb, por_liberar_7d_gb}` (calculado y guardado un minuto), `conductas` y `cola`. Probado con JUnit la forma y el orden; el disco con un volumen real no

4. Experimentos

☑ `POST /experiments` con nombre único y fecha: `201`, `400`, `404`, `409` y `500`; el creador sale del JWT y se ignora el `idInstitucional` del cuerpo
☐ `GET /experiments` con filtros `estado`, `fecha` y `tratamiento`, paginado, con `nGrupos` y `avisoVencimiento`. Hecho: `{pagina, total, items}` de 20 en 20, con `nGrupos`, `avisoVencimiento`, `creador` y los campos de la sección 12, en 4 consultas por página. Falta: los filtros en el servidor (hoy el frontend filtra en el navegador)
☑ `GET /experiments/{id}` con el árbol de grupos, tandas y videos; cada video trae `fecha`, `videoDisponible`, `idAnalisis` y `subidoPor`. Limitación: `VIDEO` no guarda quién lo subió, así que `subidoPor` es el creador del experimento
☑ El estado del experimento se calcula, no se guarda: `error` > `cancelado` > `en pausa` > `procesando` > `en cola` > `completado`, además de "sin videos" y "en espera del Día 2" (§4.8)

☑ `GET /experiments/{id}/groups?q=` sugiere grupos y la siguiente tanda (sin distinguir mayúsculas ni acentos)
☑ `PUT /experiments/{id}` con `404` y `409` por nombre repetido
☐ `PUT /experiments/{id}` solo para el creador o un Administrador (`403` a los demás; C-03, decidido por Erick el 25-oct, igual que eliminar). Hecho en código (encargo 33, compila); falta probarlo contra el backend
☑ Nombre, fecha y notas; el creador no cambia
☑ `404` si no existe y `409` si el nombre ya es de otro
☑ `DELETE /experiments/{id}`: solo el creador o un Administrador (R-08); borra el experimento completo (D-87, 23-oct) y responde `204`
☑ `404` si no existe
☑ `403` si es otro usuario (probado con una segunda cuenta Investigador en desarrollo, 23-oct)
☑ `409` si hay un análisis `procesando` o `en pausa` (probado con los dos estados: no se borró ninguna fila ni archivo). Antes bloquea con `FOR UPDATE`, de arriba hacia abajo, el experimento, sus grupos y sus tandas, para que una subida simultánea espere y falle con `409` (commit `5de79cf`, 23-oct); la concurrencia se probó con dos sesiones de `psql`, no con la API
☑ Los `en cola` salen de la cola
☑ Borra de las hojas hacia arriba, en una transacción: `SEGUNDO`, `OBSERVACION`, `ANALISIS`, `VIDEO`, `ESPECIMEN`, `TANDA`, `GRUPO`, `REPORTE`, `NOTIFICACION`, `EXPERIMENTO`. Probado con 2 grupos, 3 tandas y 3 análisis en cola: los conteos de las 10 tablas quedaron iguales a los de antes de crearlo. Comparte los borrados con "Eliminar una tanda"
☑ Borra también la carpeta `analisis/{id}/` y los archivos `videos/{id}.*` (después del commit)
☑ Todos los investigadores activos ven todos los experimentos (R-07)

5. Carga de video (§4.1, R-02, R-03, R-09)

☑ `POST /experiments/{id}/groups/{grupo}/videos` en `multipart/form-data` (probado hasta con 940 MB: unos 2 min, sin cargar el archivo en memoria)
☑ Extensión `.mp4` o `.mov` revisada en el servidor
☑ Reproducible y en horizontal, validado con ffprobe (con la marca de giro), y `422` con mensaje claro si falla
☑ `409` si la tanda ya tiene un video de esa sesión
☑ Crea el grupo si es nuevo, con `tipo` y `tratamiento`
☑ Crea la tanda si es nueva, con `nCilindros` de 2 a 4 y `ordinal`
☑ Crea los especímenes
☑ Guarda el archivo en `videos/{idVideo}.{ext}`; un rechazo o una falla no deja archivos ni filas
☑ Solo el Día 2 crea `ANALISIS` con `estado='en cola'`, `etapa='preprocesamiento'`, `nivelClasif='preciso'` y `fechaAnalisis=ahora` (D-79), y el worker real lo toma solo
☑ El Día 1 se guarda sin análisis
☑ Respuesta `201` con `{idVideo, idAnalisis, estadoAnalisis}`
☑ Nombre original del video (D-114, 20-oct): columna `VIDEO.nombreOriginal` (migración idempotente al arrancar), guardado sin la ruta del cliente; se ve en Resultados, en la Revisión y en el PDF (tabla "Videos analizados"); los videos anteriores dicen "nombre no registrado"
☑ Eliminar una tanda (D-115, 20-oct): `DELETE /experiments/{id}/tandas/{idTanda}` → `204`; solo el creador o un Administrador (`403`), `409` si su análisis está procesando o en pausa (en cola sí se borra). Una transacción borra segundos, observaciones, análisis, videos, especímenes, la tanda y los reportes del experimento; los archivos se borran después del commit. La letra queda libre. El grupo se conserva si le quedan tandas; si era la última, se borra también (23-oct, decisión de Erick: un grupo nunca queda sin tandas; el grupo se bloquea con `FOR UPDATE` antes que la tanda). Botón "Eliminar tanda" con confirmación

en el grupo; si es la única tanda, el diálogo avisa que el grupo también se elimina y la pantalla vuelve al experimento
☑ D-107 (23-oct): solo el creador sube videos; `403` ("Solo quien creó el experimento sube sus videos.") a cualquier otra cuenta, también a un Administrador. El `404` va primero. Probado con otro Investigador y con otro Administrador: sin filas ni archivos nuevos. Motivo: `VIDEO` no guarda quién lo subió y `subidoPor` es el creador del experimento

6. Contrato con el worker (§4.2 a §4.8)

☑ El backend nunca escribe `estado`, `etapa` ni `nivelClasif`, salvo al encolar, al reiniciar y en la pausa (§4.7). Revisado en todo el código (encargo 36): solo `VideoServiceImpl` (el `INSERT` con los valores de arranque), `PausaServiceImpl` (ajustar y continuar a `en cola`, cancelar a `cancelado`) y `RevisionServiceImpl.reiniciar`; no hay repositorio ni `setEstado` de `Analisis`
☑ `GET /experiments/{id}/status` con porcentaje de 0, 33, 67 o 100 por etapa. Es la única fuente del progreso: el frontend ya no usa una cola global
☐ Pausa. Hecho: diseño aprobado por Erick el 23-oct, siguiendo §4.3 de la guía, en dos encargos. El 31 (fst_auto acepta `--correccion` para medir un cilindro con el recuadro que ajusta el usuario desde el segundo de la pausa) está TERMINADO en la rama `wip/correccion-recuadro` de fst-etiquetador (commit `6b27226`; `main` de fst_auto sigue en `9844e57`) y copiado a `pipeline/` (`b22b205`); sin la opción da lo mismo que antes. El 32 se cerró el 25-oct: el worker detecta 3 s seguidos sin ver al espécimen (la racha más temprana de los últimos 300 s; en empate, el cilindro menor), pausa (primero el archivo, después el estado), guarda «no se ve» como conducta nula y al retomar acumula `correcciones.json` y llama a `etiquetar.py --correccion`; el backend tiene `GET .../pause`, `.../pause/image`, `POST .../pause/adjust` y `.../cancel` (`PausaController`). Convención: el segundo k va del instante k−1 al k; con la racha k, k+1, k+2 la foto va en k+0.5 y la corrección empieza en k−1; `ancho` y `alto` de `pausa.json` salen de la foto. 36 pruebas unitarias del worker. Prueba en desarrollo (25-oct, `Acu-YBN-1.MOV` recortado a 150 s): un cilindro tapado con color plano o con el fondo del video NO provocó la racha (el detector siguió viendo manchas), así que la pausa se forzó llamando a `pausar_analisis` con k = 71: foto en 71.5 s, 1920×1080 y 4 recuadros que caen sobre los tubos; el ajuste válido dejó `correcciones.json` con `desde_segundo` 70 y, al retomar, de 600 segundos solo cambiaron 3 del cilindro corregido (75, 99 y 133) y ninguno antes del 70. Encargo 33 (25-oct, en código sin probar contra el stack, a pedido de Erick): la foto se abre igual que fst_auto (`cv2.VideoCapture` sin tocar la orientación, como `lib.py`); regla de «recuadro casi igual» y ruta `continue` (casillas de abajo). Falta: (1) probar en la máquina de Erick el encargo 33 y la pantalla de pausa contra el backend real; (2) la racha y los segundos nulos disparados por un video real: solo con pruebas unitarias, porque en los 15 videos de `fst_auto/etiquetas` ningún espécimen dejó de verse 3 s seguidos (máximo 1 s, dos veces) y un video fabricado no la provoca
☑ `GET .../pause` lee `pausa.json` y calcula `segundosRestantes`. Probado en desarrollo (25-oct): `200` con una pausa real del worker
☑ `GET .../pause/image`. Probado (25-oct): `200` `image/jpeg`
☑ `POST .../pause/adjust` escribe `ajuste` en el JSON y cambia `en pausa` a `en cola`, solo si sigue `en pausa` (`409` si no). Probado (25-oct): una caja fuera de la imagen da `400`, sin sesión `401`, la válida `200` con `fechaAnalisis` conservada; el `409` en la ronda del 23-oct
☑ `POST .../cancel` cambia a `cancelado` con la misma condición. Probado (25-oct): `200` y el análisis queda `cancelado`
☑ Ambas solo para el creador del experimento o un Administrador (D-107: es quien subió el video); `403` a los demás, probado el 23-oct
☑ Actualización condicional atómica (`UPDATE ... WHERE estado='en pausa'`; si no cambia nada, el ajuste se deshace y responde `409`)

☐ «Recuadro casi igual» (Erick, 25-oct): si ningún borde del ajuste se movió más de `app.pausa.umbral-recuadro-px` (3 por omisión, `PAUSA_UMBRAL_RECUADRO_PX`) respecto al recuadro automático de ese cilindro en `pausa.json`, `adjust` responde `422` con el aviso. Hecho en código (encargo 33) con 4 pruebas JUnit que pasan (`PausaServiceUmbralTest`, la primera prueba de Java del backend); falta probarlo contra el backend
☐ `POST .../pause/continue` («El recuadro está bien, continuar»): mismos permisos y errores; escribe `continuar` en `pausa.json` y después pasa a `en cola` conservando `fechaAnalisis` (si no cambia nada, lo deshace y `409`). Al retomar, el worker guarda la racha en `rachas_aceptadas.json` y ya no pausa en ese tramo «no se ve» (todo el tramo seguido, aunque dure más de 3 s); otro tramo sí pausa; esos segundos quedan con conducta nula. Hecho en código (encargo 33) con pruebas unitarias del worker escritas (corren en la imagen) y 4 de vitest de la pantalla que pasan; falta probarlo contra el backend
☑ `POST .../restart` (R-04), `202` con `{idAnalisis, estado}` (encargo 36):
☑ Solo desde `completado`
☑ Solo para quien subió el video o un Administrador (`404` primero, luego `403`)
☑ En una transacción: borra `OBSERVACION` y `SEGUNDO`, borra los reportes y deja `en cola` con los valores de arranque (`etapa` y `nivelClasif` por `DEFAULT`, `rutaDiagnostico` nula, `fechaAnalisis` = la hora del reinicio, así que entra al final de la cola). Como último paso de la transacción (el worker no ve el análisis en cola hasta el commit) borra `pausa.json`, `pausa.jpg`, `correcciones.json`, `rachas_aceptadas.json` y `trabajo/`; no toca el video, `web.mp4`, `cajas.json`, `progreso.json` ni `tiempos.json` (el worker los reescribe). Probado con JUnit, incluidos dos reinicios a la vez; falta probarlo con el worker real
☑ `409` en `error`, `en pausa`, `cancelado` o en curso (también `en cola`)
☑ `409` si el video ya se borró (`VIDEO.archivo` nulo), porque reiniciar vuelve a correr el análisis sobre el video
☑ `GET .../diagnostic` entrega el PDF de diagnóstico

7. Resultados y revisión

☑ `GET .../results`, con totales y porcentaje por conducta y la bandera `revisado`
☑ `GET .../results/byminute`: cada espécimen con sus minutos (las cuatro conductas siempre), desde `SEGUNDO`
☑ `GET .../results/groups`, con promedio (no suma), datos faltantes y aviso si mezcla `preciso` y `agrupado`
☑ `GET .../results/blocks`, bloques de 5 s con la mayoritaria y el desempate entre nado y escalamiento por "conducta activa" (R-12). Regla en `BloquesDeCinco` (12 pruebas unitarias) y la ruta en `BloquesDeResultadosTest` (5 pruebas con la base): gana la conducta con más segundos; cualquier empate arriba es «conducta activa»; los cinco sin ver, «conducta activa» con matiz; matiz si algún segundo del bloque no se vio; el último bloque de un video que no mide múltiplo de 5 s trae solo los segundos que existen y su ausencia no da matiz; «conducta activa» del modelo cuenta como una conducta más (no se suma a nado ni a escalamiento)
☑ Los totales se calculan desde `SEGUNDO`, nunca se guardan (D-52)
☑ `GET .../review`, con `puedeCorregir`, recuadros desde `cajas.json` y segundos: la revisión carga con el front (probado por Vanesa en el navegador, 20-oct, backend `e66ce40`; `PARA_ERICK.md`, punto 54). `puedeCorregir` en falso con otra cuenta y sin video, y en verdadero para el creador y el Administrador, probado con JUnit (`RevisionYVideoTest`)
☐ `GET .../video` con soporte de `Range` y token de video (decidido el 15-oct). Hecho: el video con `?vt=` responde `206` con `Range` y un `web.mp4` real (probado por Vanesa en el navegador, 20-oct, backend `e66ce40`, y con JUnit); falta la casilla de los logs de nginx
☑ `POST .../analyses/{idAnalisis}/video-token`: token firmado de solo lectura de ese análisis, con vigencia `VIDEO_TOKEN_MINUTES`; solo para quien subió el video o un Administrador, con el análisis `completado` y el video disponible [PROPUESTA]. Hecho: responde `200` con `{token}` cuando lo pide el front

(Vanesa, 20-oct, backend `e66ce40`); probado con JUnit (`RevisionYVideoTest`): creador y Administrador `200`, otra cuenta `403`, sin sesión `401`, `409` si no está completado (los cinco estados) o sin video (`archivo` nulo o sin `web.mp4`), `404` si el análisis no es de ese experimento
☑ `GET .../video` acepta `?vt=` solo en esta ruta y solo para ese análisis; el JWT de sesión no se acepta en la URL y el token de video no sirve en otras rutas. Hecho: con un `vt` válido responde `206` (Vanesa, 20-oct, `e66ce40`); sin `vt` o con uno inválido da `401` (probado sin sesión); probado con JUnit: el token de otro análisis, el JWT de sesión (en la URL o en la cabecera), un token inventado y la ausencia de token dan `401`; el token de video con `Authorization: Bearer` o en `?vt=` no sirve en `/review`, `/results`, `/video-token`, `/restart` ni `/users/me` (`401`)
☐ Sin query string en los logs de nginx ni de Spring. Hecho: el `access_log` de nginx ya no guarda `?vt=` y Spring no la registra. Falta: el `error_log` de nginx sí escribe la petición completa, con `?vt=`, cuando falla el backend (hallazgo del 19-oct, pendiente)
☑ Si el video ya se borró, responde que ya no existe (`404`), con `VIDEO.archivo` nulo o sin `web.mp4`, aunque el token sea válido. Probado con JUnit
☑ `PUT .../segundos`: Hecho: responde `200` al guardar desde el front (probado por Vanesa en el navegador, 20-oct, backend `e66ce40`); las reglas de abajo están implementadas (encargo 15) y probadas con JUnit (`RevisionYVideoTest`, encargo 37)
☑ Solo los segundos que cambiaron: el backend guarda exactamente los segundos que trae el cuerpo y no toca los demás (no compara con lo que hay). Todo o nada: un error en cualquiera no guarda ninguno. Probado con JUnit
☑ `origen` en `confirmada`, `corregida` o `dudosa` (`maquina`, otro valor, vacío o sin origen dan `400`). Probado con JUnit
☑ Solo el que subió el video o un Administrador (`403` a otra cuenta, `401` sin sesión). Probado con JUnit
☑ Solo sobre un análisis `completado` (`409` en los otros cinco estados). Probado con JUnit
☑ `409` si el video ya se borró (`VIDEO.archivo` nulo): sin video no se corrige. Probado con JUnit

8. Reportes (§7, R-06)

☐ CSV con las columnas `Grupo, Tanda, Espécimen, Tiempo, Nado, Escalamiento, Inmovilidad, Conducta activa`
☐ Estadísticas por grupo: media, desviación estándar y varianza
☐ XLSX (Apache POI)
☑ PDF de resultados con iText (`GET /experiments/{id}/report/pdf`), con los minutos contados desde `SEGUNDO`
☐ Se guardan en `REPORTE` (C-08, Erick, 25-oct). Hecho en código (encargo 34, compila): `ReporteAlmacen` guarda el archivo en `reportes/{idExperimento}/experimento.pdf` (después `grupo-{id}` y `tanda-{id}` en CSV y XLSX) con escritura atómica y su fila con `INSERT ... ON CONFLICT (ruta)`; si hay fila y archivo, entrega el guardado. Con `MODO_DESARROLLADOR` no guarda nada (el PDF lleva los tiempos). Se quitó `uq_reporte_formato` (C-07, cambio de esquema aprobado por Erick; migración `DROP CONSTRAINT IF EXISTS`) para guardar los de cada grupo y tanda; `uq_reporte_ruta` se queda. 5 pruebas JUnit (`ReporteAlmacenTest`) pasan; falta probarlo contra el backend
☐ Vigencia. Hecho en código (encargo 34): en lugar de comparar fechas, cada cambio que altera lo que muestra el PDF borra las filas y, después del commit, la carpeta `reportes/{id}/`; la siguiente descarga lo regenera. Se invalida al corregir segundos, editar el experimento, eliminar una tanda o el experimento, cambiar el nombre o los apellidos en el perfil (todos los experimentos de la cuenta) y cuando el worker completa un análisis o lo pasa a `error`. No se invalida al subir un video, en pausas y cancelaciones ni al borrar videos, porque el PDF no los muestra. Contra una descarga simultánea, toda transacción que invalida bloquea PRIMERO la fila del experimento (`FOR UPDATE`), igual que la descarga: un solo orden de candados en el backend y el worker (`eliminarTanda` ahora también empieza por el experimento). La tarea

diaria `borrar_reportes_huerfanos` quita carpetas de experimentos que ya no existen. Pruebas del worker escritas (`test_reportes.py`, corren en la imagen); falta probarlo contra el backend
☐ Indican si hubo revisión humana
☐ Los reportes se conservan aunque el video se borre. Hecho en código (encargo 34): borrar un video (30 días o disco) no invalida el reporte (RN-06); falta probarlo

9. Notificaciones y correo (§11.2)

☑ `EmailService` genérico (destinatario, asunto, texto)
☑ Si el correo falla, no se detiene la operación
☑ `GET /notifications` y `PATCH /notifications/{id}` para marcar como leída. Cada notificación trae `tipo`, `mensaje`, `leido`, `fechaCreacion` e `idExperimento`; el frontend arma el título y el enlace
☑ Solo las 6 notificaciones del dominio: las crea solo el worker; el backend no agrega otras
☐ Cada notificación se envía también por correo
☑ Quitar el correo de demostración al crear un experimento, que no está en la guía

10. `worker` y tareas programadas

El worker está en `../worker` y es Python, no Spring (servicio `worker` del compose). Revisado punto por punto contra el código el 23-oct: `cola.py` (la cola, las pausas vencidas y la reconexión), `worker.py` (el análisis con el modelo), `diagnostico.py` (el PDF de error), `tareas_programadas.py`, `progreso.py`, `tiempos.py` y `vigilancia.py`. Usa el modelo `fst_cnn_seg.pt` con el código de `fst_auto` copiado sin cambios en `../pipeline`; escribe `OBSERVACION` y `SEGUNDO` (los últimos 300 s de cada espécimen) con el modelo de 12 tablas, más `web.mp4`, `cajas.json` y la notificación. Probado con `AcuBN1.MOV` (18-oct): completado, 4 × 300 segundos en `SEGUNDO`, `web.mp4`, `cajas.json`, PDF y notificación, y el camino de error con un archivo que no es video. Guarda en la base exactamente lo que da el modelo (100 %), pero el modelo en Docker coincide en 95.75 % de los segundos con el mismo modelo corrido en la PC (la diferencia nace al medir los fotogramas, antes de la red). Pruebas automáticas de `worker/` (`test_tiempos`, `test_vigilancia` y `test_diagnostico`): 23 pasan dentro de la imagen del worker (23-oct). **Pausa de R-15** (encargo 32, cerrado el 25-oct): revisa los segundos en que la segmentación de fst_auto no encontró al espécimen (`cx` vacío en `_cuadros.csv`), pausa con 3 seguidos ANTES de correr la red y guarda esos segundos con conducta nula; probada con una pausa forzada, ver la casilla «Pausa» de la sección 6. El aviso a 7 días no crea notificación: la guía (§8) dice que no se guarda nada y que el backend lo calcula (`avisoVencimiento` de `GET /experiments`); el worker solo lo deja en el log. El respaldo de la base con `pg_dump` (D-18) ya está hecho desde el 23-oct (casilla de abajo). Hasta el 20-oct hubo además un simulador (`worker_fake`, con conductas inventadas); se retiró ese día y su cola, pausas y tareas diarias pasaron a `../worker`. Agregado el 19-oct (versión 0.22.0): **progreso real** (encargo 14: `progreso.json`, `/status` con `porcentajeEtapa` y el porcentaje general, pesos medidos 1/9/90); **tiempos del análisis** (encargo 16: `tiempos.json` con segundos por etapa y el equipo, en Progreso, Resultados y el PDF); **`MODO_DESARROLLADOR`** (encargo 17: con `false` se apagan los tiempos y el banner rojo "BETA vX.Y.Z"). Probado con pruebas automáticas y sin sesión; falta una corrida real con sesión (la hará el usuario en otra PC).

☑ Toma un análisis a la vez, el más antiguo primero (`cola.py`: `tomar_siguiente`, `ORDER BY fechaanalisis, idanalisis` con `FOR UPDATE SKIP LOCKED`). Probado en desarrollo (23-oct): con dos análisis `en cola` de fechas del año 2000, `tomar_siguiente` toma el más antiguo aunque el otro tenga id menor; con ese bloqueado sin confirmar por otra conexión, la segunda conexión toma el otro y no el mismo; y dos vueltas reales de la cola (`cola.vuelta`) con un procesamiento simulado de 6 s muestran un solo análisis `procesando` a la vez
☑ Caso exitoso: pasa por las etapas (`preprocesamiento`, `cilindros`, `clasificacion`), escribe `OBSERVACION` y `SEGUNDO` (últimos 300 s, renumerados) y crea la notificación `analisis completado`. Probado con `AcuBN1.MOV` (18-oct); en el portable hay dos análisis completados (19-oct) con su notificación

☑ Caso de error (el video no se abre, un paso del pipeline falla o pasa de una hora, no coinciden los cilindros, o un fallo imprevisto, D-88): PDF de diagnóstico, estado `error` y notificación. Probado con el video que no se puede abrir; los demás motivos comparten `marcar_error` y no se provocaron a mano
☑ PDF de diagnóstico y estado `error`: probado con un archivo que no es video (18-oct), con `worker/test_diagnostico.py` (6 pruebas, 23-oct) y en desarrollo el 23-oct (video sustituido por basura: `error` en `preprocesamiento`, `diagnostico.pdf` de 80 KB válido, `rutaDiagnostico` guardada, `/status` en `error` y `GET .../diagnostic` con `200` `application/pdf`)
☑ Notificación `error del pipeline`: probado en desarrollo (23-oct), una fila para el creador con la etapa y el motivo («No se pudo leer el video con ffprobe…»). El correo no salió porque MailHog no estaba levantado; el aviso queda en la campanita
☐ Pausa (R-15). Hecho (encargo 32, 25-oct; ver la casilla «Pausa» de la sección 6): detección de 3 s seguidos sin animal, pausa y reanudación con correcciones, y conducta nula en los segundos «no se ve»; 36 pruebas unitarias y una pausa forzada en desarrollo con el pipeline real al retomar. Falta abrir la foto igual que fst_auto y la regla de «recuadro casi igual» (encargo 33)
☑ Vencimiento a las 72 horas (D-112), `cola.py`: `revisar_pausas`. Probado en desarrollo (23-oct) con `pausa.json` escritos a mano: el de 96 h pasa a `cancelado` con la notificación `analisis cancelado`; el que no tiene `pausa.json` también se cancela (solo un aviso en el log, sin notificación); el de 96 h con `ajuste` y el de 1 h siguen `en pausa`
☑ Ajuste y cancelación del usuario: el worker lee `ajuste` al retomar, lo pasa a `correcciones.json` (desde k−1) y borra `pausa.json` y `pausa.jpg`. Probado en desarrollo (25-oct) con el ajuste y la cancelación del backend
☑ Al arrancar, reencola los `procesando` huérfanos (D-84), `cola.py`: `reencolar_huerfanos`. Probado en desarrollo (23-oct), llamándola como lo hace `main()` al conectarse: los análisis `procesando` vuelven a `en cola` en la etapa `preprocesamiento` y conservan su `fechaAnalisis`
☑ Reemplaza `fechaAnalisis` por `ahora` al terminar (`completar_real` y `marcar_error`). Probado en el portable (19-oct): los análisis 1 y 3, completados, tienen `fechaAnalisis` 4 min 57 s y 4 h 21 min después de subir el video; el análisis cancelado (el 2) conserva la hora de entrada
☑ Tareas diarias en un módulo aparte, disparables a mano: `tareas_programadas.py` (`python tareas_programadas.py`, o `--disco` / `--respaldo` para una sola). Corrieron completas en desarrollo (23-oct) y en el portable
☑ Borrar videos vencidos a los 30 días (R-05), `borrar_videos_vencidos`. Probado en desarrollo (23-oct) con `python tareas_programadas.py`: un análisis completado con `fechaAnalisis` de hace 31 días pierde `videos/{id}.mp4` y `web.mp4`, `VIDEO.archivo` queda en nulo y sus 5 `SEGUNDO` se conservan. No crea notificación (la de `video borrado` solo sale en la limpieza por disco)
☑ Borrar `pausa.json` y `pausa.jpg` huérfanos (D-89, regla c), `borrar_pausas_huerfanas`. Probado en desarrollo (23-oct): se borran los de un análisis cancelado y uno completado; se conservan los de dos análisis `en pausa`
☑ Aviso a 7 días: resuelto (D-45). La guía §8 dice que no se guarda nada y que el panel calcula el aviso con la misma regla, así que no hay notificación. Lo calcula el backend en `avisoVencimiento` de `GET /experiments` (`ExperimentoServiceImpl`): solo análisis `completado` con el video todavía disponible (`archivo` no nulo) y días restantes = 30 − días desde `fechaAnalisis` (con `FLOOR`) ≤ 7, que es lo mismo que `fechaAnalisis + 30 días ≤ ahora + 7 días`; trae `fechaBorrado`. Verificado en el código (23-oct), sin prueba con datos de ese rango. El worker solo lo deja en el log
☑ Vigilar el disco: alerta al 80 %, borrado al 90 % (`vigilar_disco`; también corre antes de cada análisis, D-86). Probado en desarrollo (23-oct) con `DISK_SIMULATED_PERCENT`: al 85 %, una sola `alerta de disco` al administrador y la segunda corrida del mismo día no la repite; al 95 %, `liberar_disco` borra un video por corrida del más antiguo al más nuevo (49 y luego 50), salta el video más antiguo de todos porque su análisis estaba `en cola` y avisa al creador con `video borrado`. Con el porcentaje real (4.1 %) no hace nada

☑ Respaldar la base con `pg_dump` cada día, conservar los últimos 7 y borrar los más viejos (§8, D-18, agregado a la guía el 20-oct). Respalda la base, no los videos. Hecho el 23-oct: `respaldar_base()` en `tareas_programadas.py` (primera de las tareas diarias, o `--respaldo` sola), `pg_dump -Fc` 17.11 de la imagen del worker contra el servidor 16.15, con la contraseña solo en `PGPASSWORD` del subproceso; archivo `fst-AAAAMMDD.dump` en `respaldos/` (carpeta del equipo fuera de los volúmenes, bind mount en los dos compose; `.gitignore`), escrito a un temporal y renombrado al final. Probado en desarrollo: crea el archivo y `pg_restore --list` lo lee sin errores; con 8 respaldos falsos viejos y un `.tmp` quedan 7 y el `.tmp` se borra; el de hoy se reemplaza; con contraseña equivocada o sin `pg_dump` solo se registra el error (código 1, sin tocar los respaldos buenos) y las demás tareas siguen; ni la contraseña real ni una falsa aparecen en el log; 5 pruebas automáticas (`test_respaldo.py`). Restauración probada en una base temporal: mismos conteos, restricciones, índices y dominios, también una segunda vez encima (`--clean --if-exists`); hay que usar el `pg_restore` 17 de la imagen del worker, porque el 16 del contenedor de la base no lee archivos de `pg_dump` 17, y termina con un aviso benigno por `SET transaction_timeout`. Cómo restaurar: `portable/LEEME.md`
☑ `ultima_corrida_tareas.txt`: se escribe al terminar las tareas diarias y evita repetirlas el mismo día. Probado en el portable (23-oct): `worker/ultima_corrida_tareas.txt` en el volumen, con la fecha de ese día

11. Criterios de aceptación finales (§12)

☐ `docker compose up` arranca en una máquina limpia, y en una segunda máquina. Hecho: el portable se probó desde el zip el 18-oct y Erick lo usa en otra PC; falta probar el compose definitivo de la raíz
☑ Un investigador recibe `403` en `/admin/*`. Probado con JUnit (encargo 35)
☑ No se puede quitar al último Administrador. Probado con JUnit, también con dos peticiones a la vez (encargo 35)
☑ Un Día 1 se guarda sin análisis
☐ Con dos análisis en cola, el worker los procesa uno a la vez. Hecho: probado en desarrollo (23-oct) con dos vueltas reales de la cola (`cola.vuelta`) y un procesamiento simulado de 6 s: el más antiguo primero, un solo `procesando` a la vez y el segundo espera. Falta repetirlo con dos videos reales y el modelo (varios minutos y unos 3.5 GB de memoria)
☐ Un reporte se regenera si hay un análisis más nuevo. Hecho en código (encargo 34): al completar un análisis el worker invalida los reportes del experimento; falta probarlo con dos análisis reales
☐ Cuatro sesiones simultáneas responden en 3 s o menos
☐ El resumen final anota los puntos [ABIERTO] que se resolvieron con el valor por defecto

12. Decisiones por tomar con el frontend (18-oct-2026)

Vanesa conectó el frontend a la guía de la API (`PARA_ERICK.md`, puntos 41 a 52, rama `feat/frontend-v2-servicios`). Lo que sigue son formas que la guía no define y que ella propone. Cada casilla se marca cuando se decide y el endpoint correspondiente queda hecho. Sin respuesta, el frontend usa su propuesta y no se rompe.

☑ `PATCH /auth/password` y `PATCH /users/me`: cuerpo `{contrasenaActual, contrasenaNueva}` y `{nombre, apellidos, correo}`; ambos devuelven la cuenta como el login, sin `token`
☑ `POST /experiments` responde `201` con `{idExperimento}`
☑ `GET /experiments`: campos extra `nEspecimenes`, `tratamientos`, `videosDia2: {analizados, cargados, total}`, `diasRetencion` (`null` sin videos o ya borrados) y `videosBorrados`
☑ `GET /experiments/{id}/status`: incluye todos los análisis vigentes, también los `completado`
☑ `/status`: `fechaAnalisis` es la hora de entrada a la cola mientras el análisis no termina (D-79); el frontend ordena la cola con ella

☑ `/status`: campo extra `motivoCancelacion` (`usuario` o `plazo`). Decidido (D-112): no se guarda; el backend manda `null` y el frontend dice «El análisis se canceló.» (commit `3d9ccec` de Vanesa)
☐ `/status`: campo extra `pausa.desdeSegundo`. Hecho: el backend lo lee de `pausa.json` (`ResultadosServiceImpl.pausa`); el worker ya crea pausas (25-oct), pero no se ha comprobado este campo en `/status` con una (ver la casilla «Pausa» de la sección 6)
☑ `GET /experiments/{id}`: cada video trae `fecha`; `/status` trae `fechaAnalisis` por análisis
☑ `GET /admin/system`: forma `{disco, conductas, cola: [{idAnalisis, idExperimento, experimento, grupo, letra, sesion, estado}]}`, en orden de llegada (el que se procesa primero). `conductas` trae los nombres de la base y el frontend los convierte
☑ `GET /experiments/{id}/groups?q=`: forma `[{idGrupo, etiqueta, tipo, tratamiento, siguienteOrdinal}]`
☑ `{grupo}` en la URL de subida es la etiqueta del grupo, codificada (ver `services/groups.js` del frontend)
☑ `/results`: cada espécimen trae además `idGrupo`
☑ `/results/groups`: forma `[{idGrupo, etiqueta, tipo, tratamiento, n, nTotal, faltantes, nivelMixto, promedios, porNivel}]`
☑ `/results/blocks`: cada espécimen con `bloques: [{inicio, conducta, matiz, segundos}]` (y `tanda`, `ordinal`, `numeroCilindro`, `idGrupo`)
☑ `GET .../review`: agregar `duracion`, `ancho` y `alto` (sin ellos el frontend asume 300 s y 1920 × 1080): los trae (`duracion` = segundos guardados; `ancho` y `alto` de `cajas.json`) y la revisión carga con el front (probado por Vanesa en el navegador, 20-oct, backend `e66ce40`)
☑ Notificaciones: el id se llama `idNotificacion`; «marcar todas» se hace con un `PATCH` por notificación, sin `POST /notifications/read-all`
☑ `POST .../video-token` responde `{token}` y el video con `?vt=` admite `Range` con un `web.mp4` real: `200` con el token y `206` con `Range` (probado por Vanesa en el navegador, 20-oct, backend `e66ce40`)